Inteligentna wirtualizacja systemów i automatyzacja procesów

Raport z projektu "System wizyjny dla schroniska umożliwiający automatyczną identyfikację zwierząt."

Jan Rybarz i Maciej Mierkiewicz

5 czerwca 2025

1 Abstrakt

Niniejszy raport dokumentuje projekt "System wizyjny dla schroniska umożliwiający automatyczną identyfikację zwierząt", którego celem była budowa inteligentnego systemu do automatycznego rozpoznawania gatunku zwierzęcia (kot, pies, zwierzę dzikie) na podstawie zdjęć. Rozwiązanie oparto na technologii konwolucyjnych sieci neuronowych (CNN), wykorzystując biblioteki TensorFlow i Keras do implementacji modeli oraz PyQt6 do stworzenia interfejsu graficznego użytkownika (GUI). System umożliwia zarówno klasyfikację pojedynczych obrazów, jak i zarządzanie procesem trenowania własnych modeli. W ramach projektu przeprowadzono serię eksperymentów badających wpływ kluczowych hiperparametrów, takich jak rozdzielczość obrazu, liczba warstw konwolucyjnych, funkcja aktywacji, optymalizator, rozmiar partii wsadowej oraz obecność szumu w danych, na dokładność i czas treningu modelu. Analiza wyników pozwoliła na zidentyfikowanie konfiguracji oferujących optymalny kompromis między wydajnością a jakością predykcji, wskazując na model 3-warstwowy z obrazami o rozdzielczości 64x64 pikseli i rozmiarem partii 16 jako efektywne rozwiązanie.

Spis treści

1 Abstrakt **2**

2 Wybór tematu projektu **4**
2.1 Opis problemu 4

3 Studia literaturowe **4**

4 Baza danych **4**

5 Koncepcja rozwiązania **4**
5.1 Główne komponenty systemu 4
5.2 Architektura modelu i proces uczenia 5
5.2.1 Architektura modelu CNN 5
5.2.2 Proces treningu modelu 5
5.2.3 Mechanizm wczesnego zatrzymywania (Early Stopping) 6

6 Środowisko programowe **6**

7 Stworzenie oprogramowania **6**

8 Weryfikacja oprogramowania **6**

9 Analiza wyników **7**
9.1 Metodyka przeprowadzania eksperymentów 7
9.2 Seria eksperymentów 7

10 Tabela wyników eksperymentów – pełna seria **8**

11 Tabela wyników eksperymentów – najszybszy, najdokladniejszy, optymalny” **8**

12 Wizualizacja wyników eksperymentów **9**
12.1 Analiza wybranych modeli: best, fast, optimal 9
12.2 Wpływ funkcji aktywacji 10
12.3 Wpływ rozmiaru batch size 11
12.4 Wpływ rozdzielczości obrazu 13
12.5 Wpływ liczby warstw konwolucyjnych 14
12.6 Wpływ szumu w danych 17
12.7 Porównanie optymalizatorów 17

13 Wnioski **18**

14 Załączniki **18**

2 Wybór tematu projektu

2.1 Opis problemu

Wiele schronisk dla zwierząt boryka się z problemem sprawnej identyfikacji i zarządzania danymi dotyczącymi przebywających w nich zwierząt. Ręczna klasyfikacja zdjęć jest procesem czasochłonnym i obarczonym błędami ludzkimi. Niniejszy projekt ma na celu stworzenie inteligentnego systemu opartego na technologii głębokiego uczenia (CNN), który automatycznie rozpoznaje gatunek zwierzęcia na zdjęciu (kot, pies lub zwierzę dzikie). Pozwoli to na usprawnienie pracy w schroniskach oraz skuteczniejszą automatyzację systemów monitoringu i zarządzania zasobami.

3 Studia literaturowe

W ramach badań literaturowych przeprowadzono analizę dostępnych źródeł dotyczących następujących zagadnień:

- konwolucyjne sieci neuronowe (CNN) – podstawy działania i przykłady (LeNet, AlexNet, ResNet),
- warstwy konwolucyjne – mechanizm działania, pooling, normalizacja,
- funkcje aktywacji – ReLU, tanh, ELU i ich wpływ na uczenie modelu,
- optymalizatory – Adam, SGD, Adagrad,
- metryki klasyfikacyjne – Accuracy, Precision, Recall, F1-score, analiza macierzy pomyłek,
- wykorzystywane biblioteki – TensorFlow, Keras, OpenCV, PyQt6.

4 Baza danych

Projekt oparto o specjalnie przygotowany zestaw danych, który został podzielony na trzy zbiory:

- Treningowy `train`, – zestaw obrazów do uczenia modelu.
- Walidacyjny `val`, – do oceny działania modelu podczas uczenia i doboru parametrów.
- Testowy `test` – finalna ewaluacja gotowego modelu.

Każdy z tych zestawów zawiera obrazy zwierząt w trzech kategoriach: koty, psy oraz dzikie zwierzęta. Wszystkie obrazy zostały zbilansowane w celu uniknięcia problemów związanych z nierówną reprezentacją klas. Do bilansowania danych wykorzystano autorską funkcję `get_balanced_subset_and_remainder()`.

5 Koncepcja rozwiązania

Aplikacja została zaprojektowana w architekturze składającej się z dwóch głównych komponentów:

5.1 Główne komponenty systemu

- Frontend (GUI) – interfejs użytkownika wykonany w bibliotece PyQt6, umożliwiający łatwe przeprowadzanie klasyfikacji oraz trenowanie sieci neuronowych.
- Backend – logiczna warstwa aplikacji, obejmująca:
  - Moduł klasyfikujący `classifier.py`,
  - Moduł do trenowania i oceny modeli `model.py`,
  - Moduł logujący wyniki eksperymentów `experiment_logger.py`,
  - Moduł do obsługi zbiorów danych `dataset_loader.py`.

System eksportuje wytrenowane modele do formatu .keras oraz zapisuje szczegółowe logi z procesu treningu do plików .history.json.

5.2 Architektura modelu i proces uczenia

Kluczowym elementem systemu jest model konwolucyjnej sieci neuronowej (CNN) odpowiedzialny za klasyfikację obrazów. Jego architektura oraz proces treningu zostały starannie zaprojektowane i zaimplementowane w celu osiągnięcia jak najlepszych wyników.

5.2.1 Architektura modelu CNN

Podstawowa architektura modelu CNN, implementowana za pomocą funkcji `create_model` w module `model.py`, została zaprojektowana jako model sekwencyjny w bibliotece Keras. Składała się z następujących kluczowych bloków:

- **Warstwa wejściowa (Input):** Przyjmuje obrazy o zdefiniowanej rozdzielczości (np. 128x128 pikseli, konfigurowalne przez parametr `img_size`) i trzech kanałach kolorów (RGB).
- **Bloki konwolucyjne:** Rdzeń modelu stanowi sekwencja N (gdzie N to liczba warstw konwolucyjnych, np. 1, 3, 5, definiowana przez `num_conv_layers`) bloków. Każdy blok typowo zawiera:
  - Warstwę konwolucyjną (`Conv2D`) z filtrami o rozmiarze 3x3. Liczba filtrów w kolejnych warstwach zazwyczaj rośnie (np. 32, 64, 128), co pozwala modelowi uczyć się coraz bardziej złożonych cech. Domyślną funkcją aktywacji w tych warstwach jest ReLU, choć w ramach eksperymentów testowano również ELU oraz tanh.
  - Warstwę MaxPooling (`MaxPooling2D`) o rozmiarze pula 2x2, która redukuje wymiary przestrzenne map cech, pomagając w generalizacji i zmniejszeniu liczby parametrów.
- **Warstwa spłaszczająca (Flatten):** Przekształca wielowymiarowe mapy cech pochodzące z ostatniej warstwy konwolucyjnej/pooling w jednowymiarowy wektor.
- **Warstwy gęsto połączone (Dense):** Po spłaszczeniu następuje zazwyczaj jedna lub więcej warstw gęsto połączonych. W typowej konfiguracji była to warstwa ze 128 neuronami i funkcją aktywacji (np. ReLU).
- **Warstwa wyjściowa (Output Dense):** Ostatnia warstwa gęsto połączona, posiadająca liczbę neuronów równą liczbie klasyfikowanych kategorii (w tym projekcie 3: kot, pies, zwierzę dzikie). Wykorzystuje funkcję aktywacji `softmax`, która normalizuje wyjścia neuronów do postaci rozkładu prawdopodobieństwa, wskazując prawdopodobieństwo przynależności obrazu do każdej z klas.

5.2.2 Proces treningu modelu

Proces uczenia modelu CNN przebiegał następująco:

- **Przygotowanie danych:** Obrazy wejściowe były normalizowane poprzez podzielenie wartości pikseli przez 255.0, co sprowadzało je do zakresu [0, 1]. Etykiety klas były reprezentowane jako liczby całkowite (0 dla kota, 1 dla dzikiego zwierzęcia, 2 dla psa).
- **Funkcja straty (Loss Function):** Do oceny błędu modelu podczas treningu wykorzystano funkcję `sparse_categorical_crossentropy`. Jest ona odpowiednia dla problemów klasyfikacji wieloklasowej, gdzie etykiety są podane jako liczby całkowite, a nie w formie one-hot encoding.
- **Optymalizator (Optimizer):** Domyślnym optymalizatorem był Adam, znany ze swojej efektywności i adaptacyjnego współczynnika uczenia. W ramach eksperymentów testowano również inne optymalizatory, takie jak SGD (Stochastic Gradient Descent) i Adagrad, aby zbadać ich wpływ na szybkość zbieżności i jakość końcowego modelu.
- **Metryka oceny (Metrics):** Główną metryką używaną do monitorowania wydajności modelu podczas treningu i jego finalnej oceny była dokładność (`accuracy`), czyli stosunek poprawnie sklasyfikowanych obrazów do ich całkowitej liczby.
- **Walidacja (Validation):** Po każdej epoce treningowej model był oceniany na wydzielonym zbiorze walidacyjnym. Wyniki na tym zbiorze (dokładność walidacyjna `val_accuracy` i strata walidacyjna `val_loss`) pozwalały monitorować, jak dobrze model generalizuje na danych, których nie widział podczas treningu, i były kluczowe dla mechanizmu wczesnego zatrzymywania.

5.2.3 Mechanizm wczesnego zatrzymywania (Early Stopping)

Aby zapobiec przeuczeniu (overfitting) modelu, czyli sytuacji, w której model doskonale radzi sobie na danych treningowych, ale słabo generalizuje na nowe, nieznane dane, oraz aby oszczędzić czas i zasoby obliczeniowe, zaimplementowano własny mechanizm wczesnego zatrzymywania (`CustomEarlyStopping`). Działa on w następujący sposób:

- Monitoruje stratę na zbiorze walidacyjnym (`val_loss`) przez określoną liczbę kolejnych epok (parametr `patience`, domyślnie 5, zdefiniowany w `constants.py`).
- Porównuje średnią wartość `val_loss` z ostatnich `patience` epok ze średnią `val_loss` z `patience` epok bezpośrednio je poprzedzających (tj. okno przesunięte o jedną epokę wstecz).
- Jeśli różnica tych średnich wartości wskazuje na pogorszenie generalizacji (tj. strata na zbiorze walidacyjnym zaczyna rosnąć) i to pogorszenie przekracza zdefiniowany minimalny próg (`min_delta`, domyślnie 0.01), proces treningu jest automatycznie przerywany.
- Informacje o ewentualnym zadziałaniu mechanizmu wczesnego zatrzymywania, w tym epoka, w której nastąpiło zatrzymanie, oraz przyczyna, są logowane i zapisywane w pliku `.history.json` powiązanym z trenowanym modelem.

6 Środowisko programowe

Projekt wykorzystuje język programowania Python w wersji 3.10 oraz biblioteki:

- TensorFlow (uczenie maszynowe),
- OpenCV-Python (przetwarzanie obrazów),
- PyQt6 (interfejs graficzny),
- Matplotlib (wizualizacja danych),
- Pillow (obsługa obrazów).

Aplikacja działa na systemach operacyjnych Windows 11 oraz Linux i może być uruchomiona zarówno na CPU, jak i GPU z obsługą CUDA.

7 Stworzenie oprogramowania

Podczas tworzenia aplikacji zrealizowano następujące zadania:

- implementacja modułów do trenowania i klasyfikacji obrazów,
- przygotowanie interfejsu GUI pozwalającego na łatwe zarządzanie procesami treningowymi oraz ocenę modeli,
- zaprojektowanie modułu logowania eksperymentów, który przechowuje pełną historię parametrów, metryk oraz wyników,
- integracja modułów oraz przeprowadzenie kompleksowych testów.

8 Weryfikacja oprogramowania

Aplikacja przeszła kompleksową weryfikację, w skład której weszły:

- testy jednostkowe modułów backendowych,
- testy poprawności klasyfikacji na ręcznie wybranych zdjęciach,
- testowanie interfejsu graficznego pod kątem funkcjonalności i użyteczności.

9 Analiza wyników

9.1 Metodyka przeprowadzania eksperymentów

W celu zapewnienia wiarygodności i powtarzalności uzyskanych wyników, wszystkie eksperymenty przeprowadzono według ustandaryzowanej metodyki, opisanej poniżej.

- **Spójność zbioru danych:** Wszystkie warianty modeli były trenowane i testowane na tym samym, specjalnie przygotowanym zbiorze danych, podzielonym na podzbiory treningowy (`train`), walidacyjny (`val`) i testowy (`test`). Aby zagwarantować, że w każdym eksperymencie używane są dokładnie te same obrazy dla danej konfiguracji, ścieżki do plików graficznych były zapisywane w dedykowanym pliku `animal_model.dataset.json` (generowanym na podstawie `DATASET_JSON_PATH` z `constants.py`). Wykorzystano autorską funkcję `get_balanced_subset_and_remainder()` z modułu `dataset_loader.py` do zapewnienia zbilansowanej reprezentacji poszczególnych klas zwierząt, co minimalizuje ryzyko stronniczości modelu wynikającej z nierównej liczebności klas.

- **Wielokrotne przebiegi i ocena stabilności:** Dla każdej badanej konfiguracji hiperparametrów proces trenowania i ewaluacji modelu był powtarzany 20 razy (zgodnie z parametrem `RUNS` w `constants.py`). Taka procedura pozwoliła na ocenę stabilności modelu – jak bardzo jego wyniki różnią się między kolejnymi, niezależnymi procesami uczenia przy tych samych ustawieniach. Wyniki te agregowano, obliczając średnią dokładność, medianę oraz odchylenie standardowe dokładności, co dało pełniejszy obraz wydajności niż pojedynczy przebieg.

- **Szczegółowe logowanie:** Każdy eksperyment był dokładnie dokumentowany. Parametry konfiguracyjne, historia procesu treningu (metryki dokładności i straty dla każdej epoki, zarówno na zbiorze treningowym, jak i walidacyjnym), informacje o ewentualnym zadziałaniu mechanizmu wczesnego zatrzymywania oraz finalne wyniki testowe (w tym macierz pomyłek i szczegółowe metryki jakości dla każdej klasy, takie jak precyzja, czułość, F1-score) były zapisywane do plików `.history.json`. Zagregowane wyniki z serii eksperymentów (np. średnie wartości dla 20 przebiegów) zapisywano do plików `.csv` (np. `experiments_results.csv` oraz `experiments_results_3_Best.csv`), co ułatwiało późniejszą analizę i porównanie modeli.

- **Kontrolowana zmiana parametrów:** Podczas badania wpływu poszczególnych hiperparametrów (np. rozdzielczości obrazu, liczby warstw konwolucyjnych, funkcji aktywacji, optymalizatora, rozmiaru partii) na działanie modelu, starano się modyfikować tylko jeden parametr na raz, utrzymując pozostałe na ustalonym, bazowym poziomie. Pozwoliło to na bardziej precyzyjną ocenę indywidualnego wkładu każdego z badanych czynników w ogólną wydajność i charakterystykę modelu.

9.2 Seria eksperymentów

W celu znalezienia optymalnej konfiguracji modelu, przeprowadzono trzy szczegółowe eksperymenty:

- Model „fast” – 1 warstwa, rozdzielczość 32x32, batch size 8 – bardzo szybki trening przy niższej dokładności,

- Model „best accuracy” – 5 warstw, rozdzielczość 128x128, batch size 2048 – najwyższa osiągnięta dokładność przy najdłuższym czasie treningu,

- Model „optimal” – 3 warstwy, rozdzielczość 64x64, batch size 16 – najlepszy kompromis pomiędzy czasem treningu a dokładnością.

Uzyskane wyniki zestawiono w tabeli porównawczej (patrz Tabela 1 i Tabela 2) oraz na wykresach (patrz Rysunki 1 do 19).

Legenda skrótów tabel

Res Rozdzielczość wejściowych obrazów (np. 128x128 pikseli),

N Obecność szumu w danych (yes – tak, no – nie),

CL Liczba warstw konwolucyjnych,

Act Funkcja aktywacji (ReLU, ELU, tanh),

Opt Optymalizator (Adam, SGD, Adagrad),

BS Batch size – liczba próbek w jednej partii treningowej,

Acc$_{avg}$ Średnia dokładność modelu (w %),

Acc$_{med}$ Mediana dokładności,

σ_{acc} Odchylenie standardowe dokładności,

Time [s] Średni czas treningu modelu (sekundy).

10 Tabela wyników eksperymentów – pełna seria

Res	N	CL	Act	Opt	BS	Acc_{avg}	Acc_{med}	σ_{acc}	Time [s]
128x128	no	3	relu	adam	1	95.13	95.3	1.02	1594.31
128x128	no	3	relu	adam	8	95.31	95.55	1.3	580.57
128x128	no	1	relu	adam	16	92.46	92.36	1.26	333.52
128x128	no	2	relu	adam	16	94.27	94.78	1.95	386.98
128x128	no	3	relu	adam	16	96.4	96.44	0.72	651.69
32x32	no	3	relu	adam	16	94.79	94.84	0.9	61.79
64x64	no	3	relu	adam	16	95.92	95.98	0.7	221.53
256x256	no	3	relu	adam	16	95.27	95.46	0.85	1966.82
128x128	no	3	relu	sgd	16	96.1	96.26	0.93	1588.93
128x128	no	3	tanh	adam	16	40.38	33.33	17.46	1295.04
128x128	no	3	elu	adam	16	88.35	94.6	18.93	748.2
128x128	no	3	relu	adam	16	96.0	96.41	1.31	697.39
128x128	no	3	relu	adagrad	16	96.08	96.47	1.23	3287.16
128x128	yes	3	relu	adam	16	95.76	95.95	0.59	525.58
128x128	no	4	relu	adam	16	96.64	96.59	0.79	2067.85
128x128	no	5	relu	adam	16	97.04	97.15	0.63	931.51
128x128	no	3	relu	adam	32	95.95	96.16	0.88	1216.7
128x128	no	3	relu	adam	64	96.08	95.95	0.94	676.25
128x128	no	3	relu	adam	128	96.43	96.38	0.75	886.94
128x128	no	3	relu	adam	2048	96.99	96.99	0.53	2994.12
128x128	no	3	relu	adam	4096	96.36	96.78	1.35	1935.86

Tabela 1: Porównanie modeli z eksperymentów (`experiments_results.csv`).

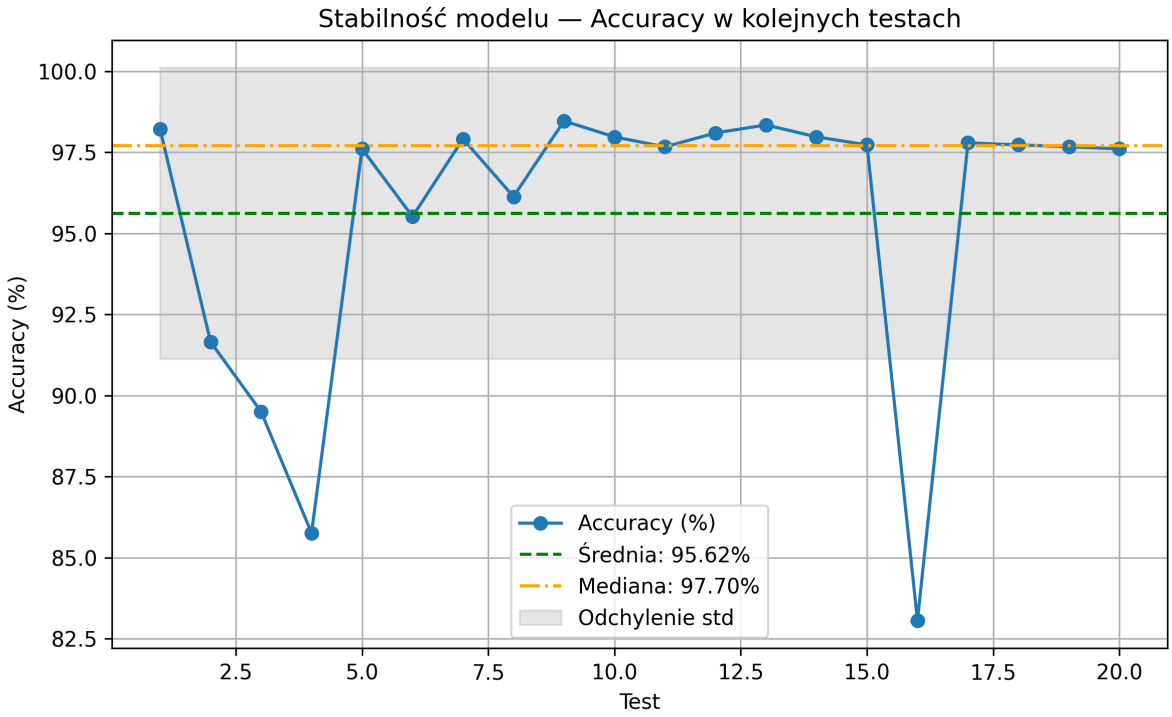
11 Tabela wyników eksperymentów – najszybszy, najdokładniejszy, optymalny”

Res	N	CL	Act	Opt	BS	Acc_{avg}	Acc_{med}	σ_{acc}	Time [s]
32x32	yes	1	relu	adam	8	92.35	92.39	1.00	67.2
128x128	no	5	relu	adam	2048	95.62	97.70	4.49	1879.92
64x64	no	3	relu	adam	16	95.99	96.10	0.62	184.49

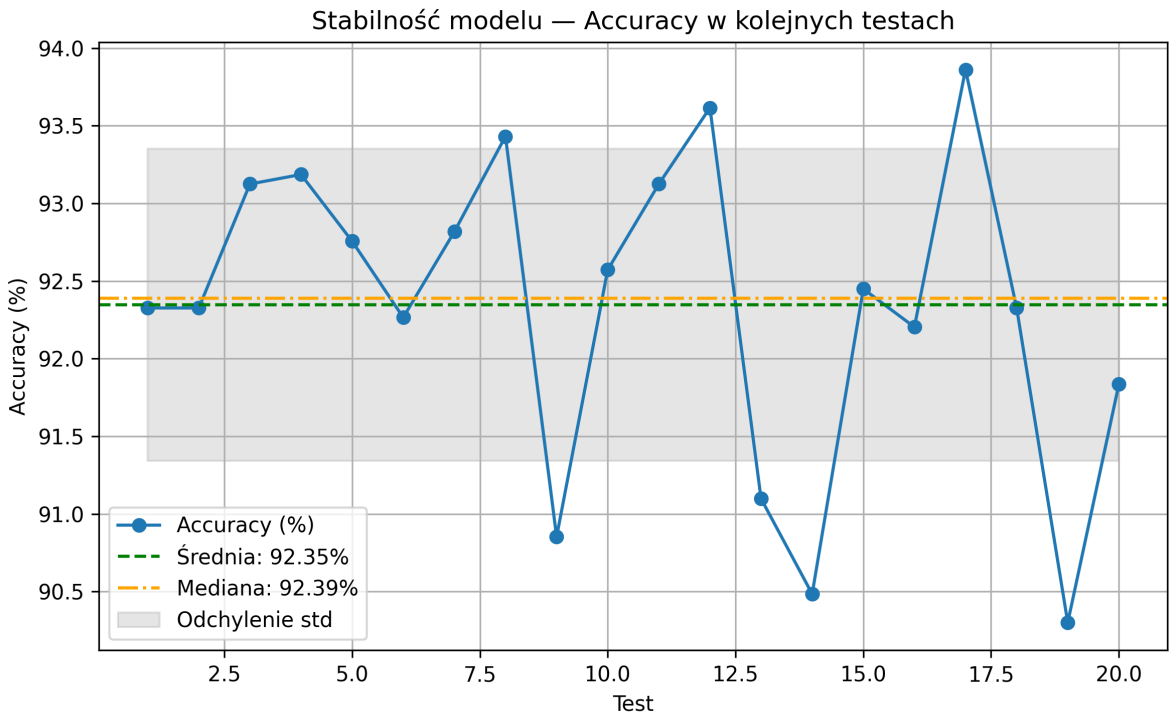
Tabela 2: Porównanie modeli z eksperymentów (`experiments_results_3_Best.csv`).

12 Wizualizacja wyników eksperymentów

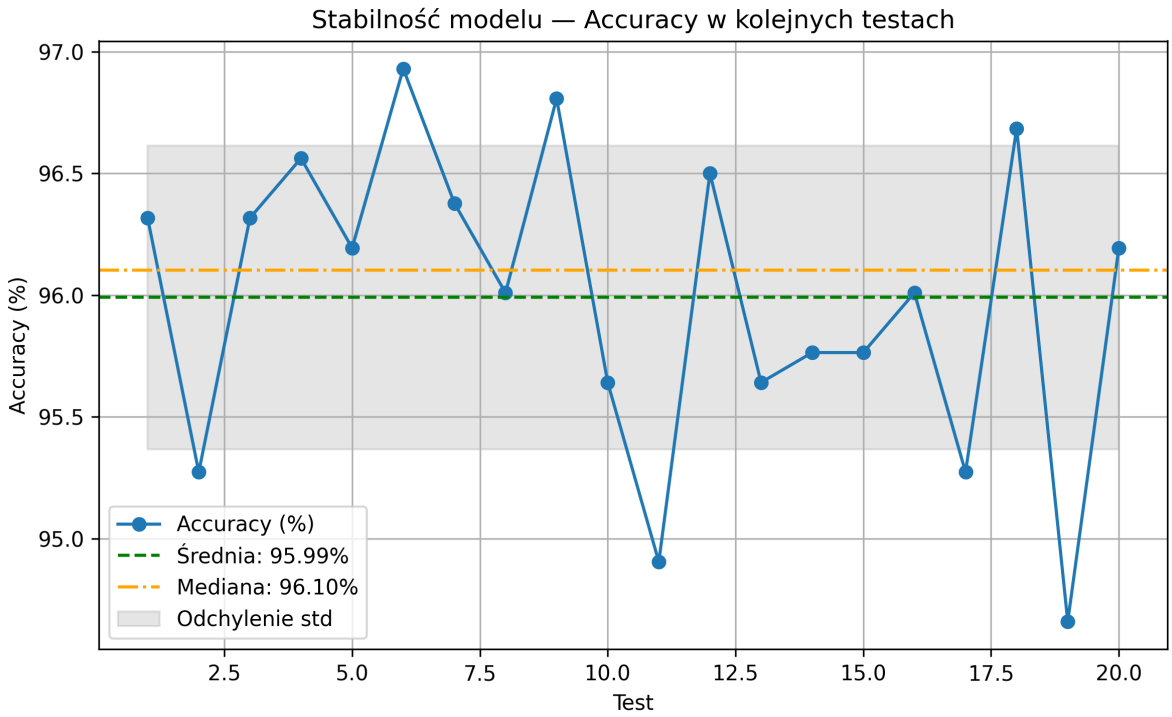
12.1 Analiza wybranych modeli: best, fast, optimal



Rysunek 1: Model z najwyższą osiągniętą dokładnością (5 warstw, 128x128, BS = 2048).

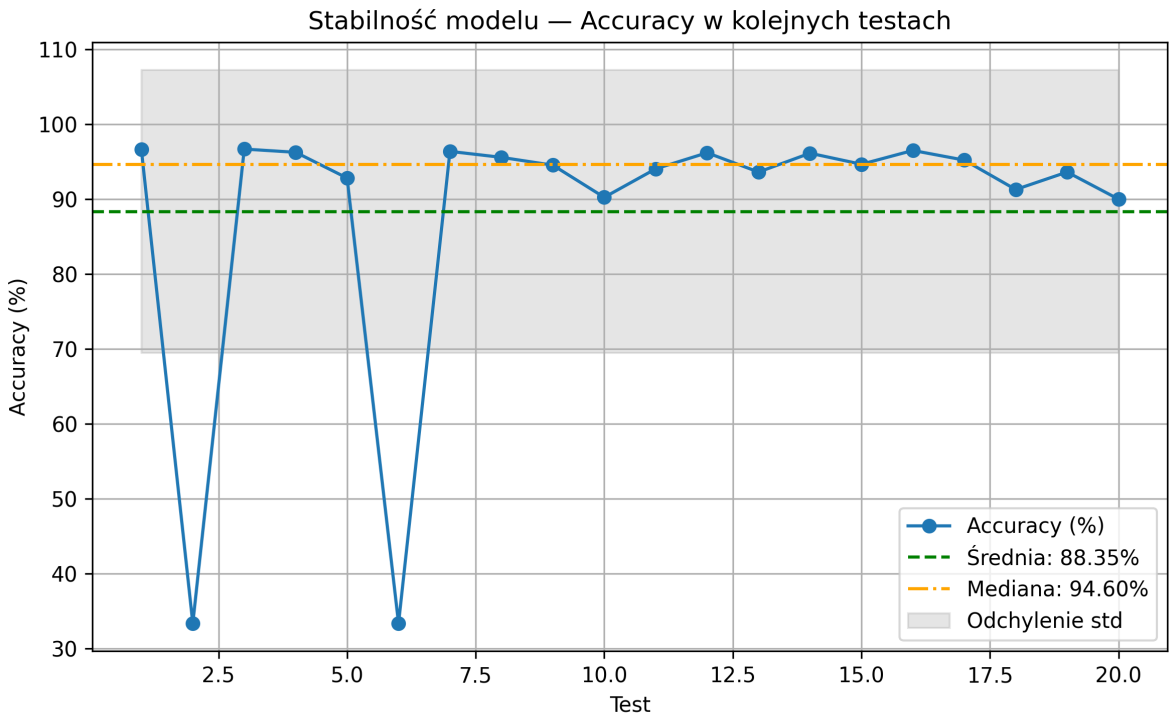


Rysunek 2: Model „fast” – bardzo szybki trening kosztem dokładności.

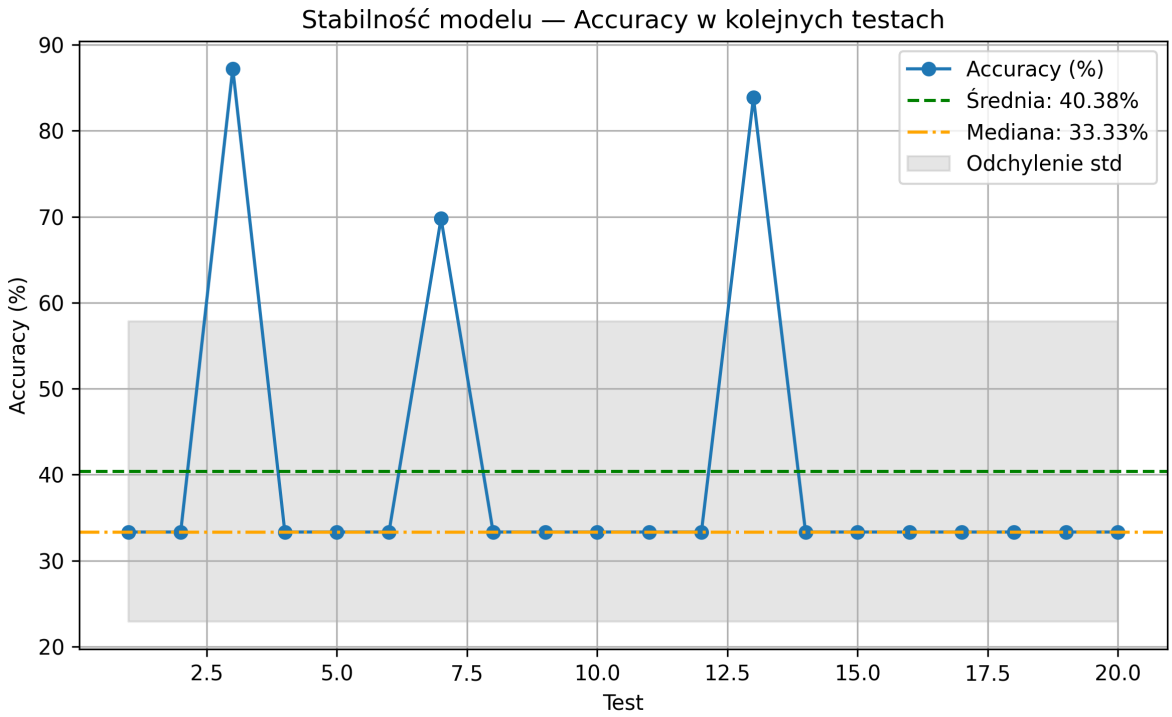


Rysunek 3: Model „optimal” – najlepszy kompromis między wydajnością a dokładnością (3 warstwy, 64x64, BS = 16).

12.2 Wpływ funkcji aktywacji

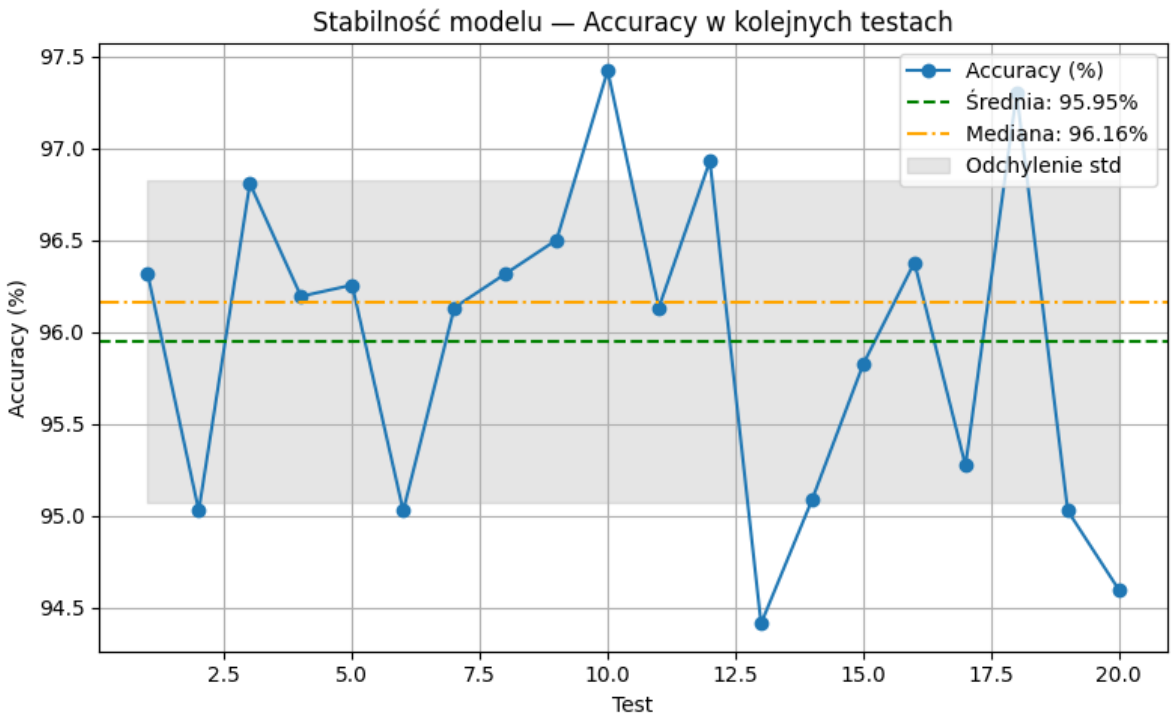


Rysunek 4: Wpływ funkcji aktywacji ELU na stabilność i dokładność modelu.

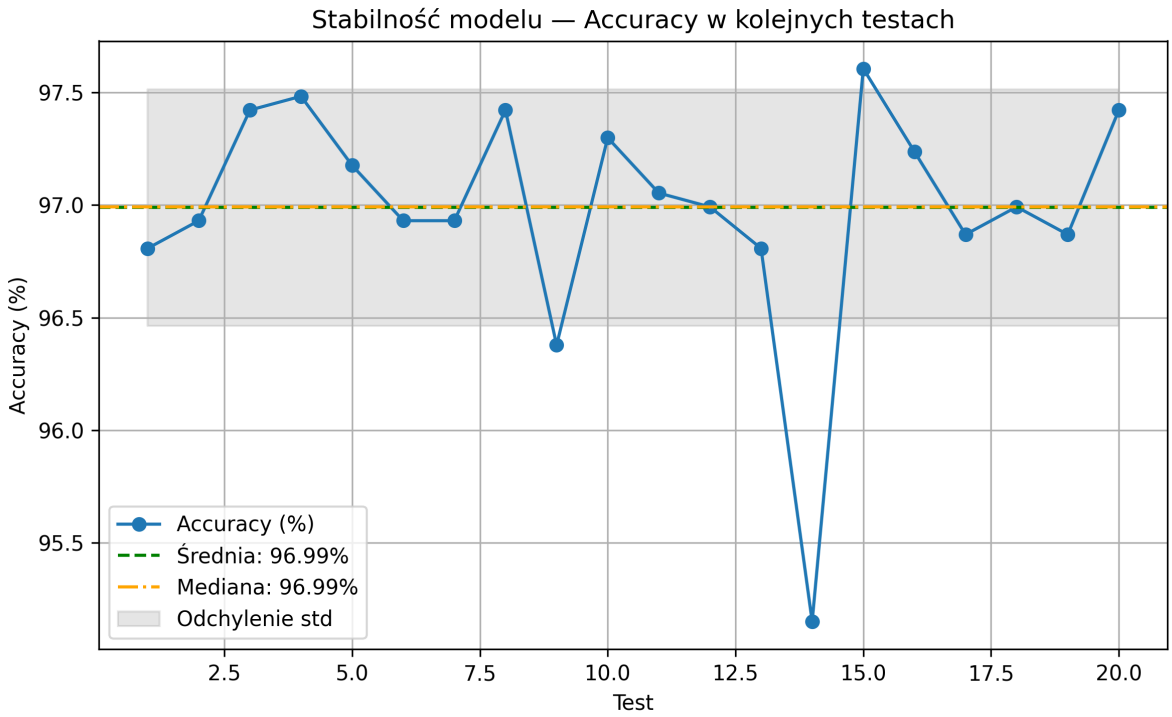


Rysunek 5: Wpływ funkcji aktywacji tanh na proces uczenia modelu.

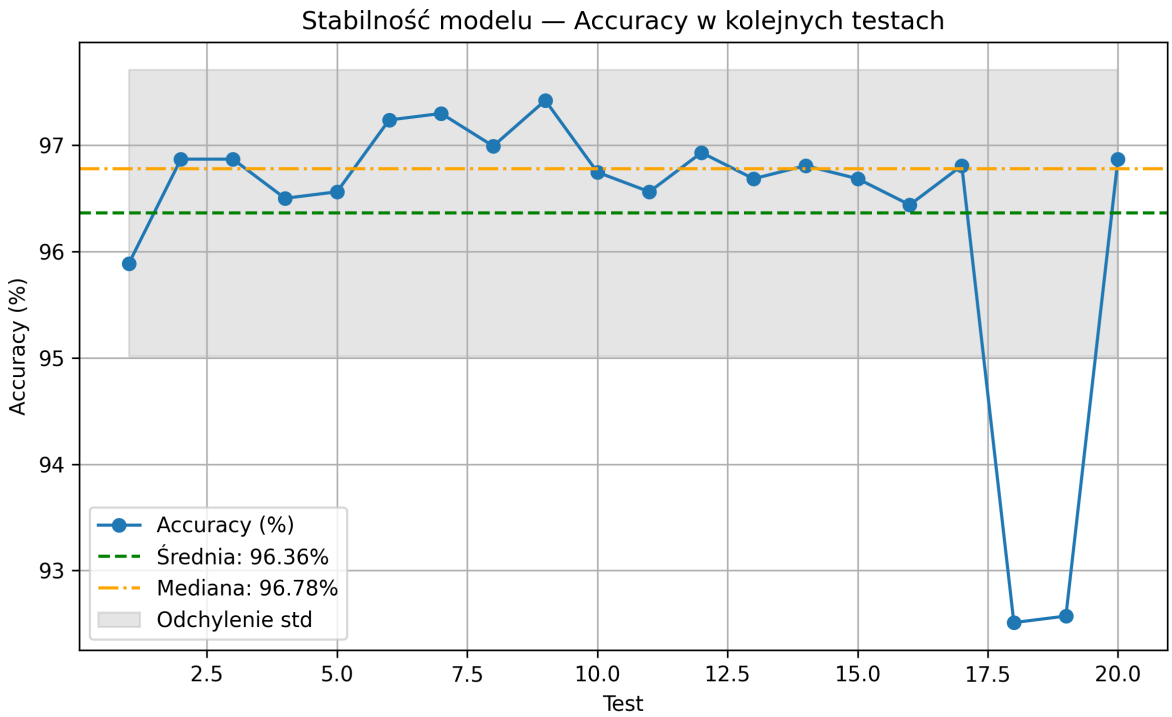
12.3 Wpływ rozmiaru batch size



Rysunek 6: Stabilność treningu dla batch size = 32.

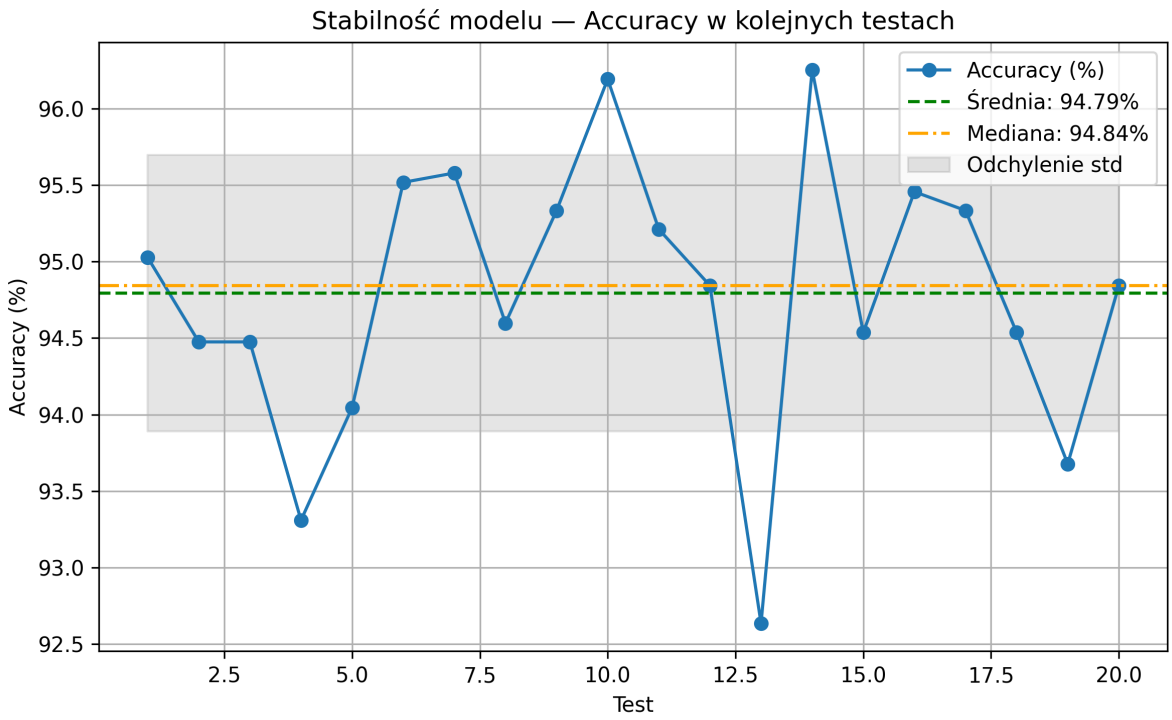


Rysunek 7: Wpływ bardzo dużej partii danych (batch size = 2048) na czas i jakość uczenia.

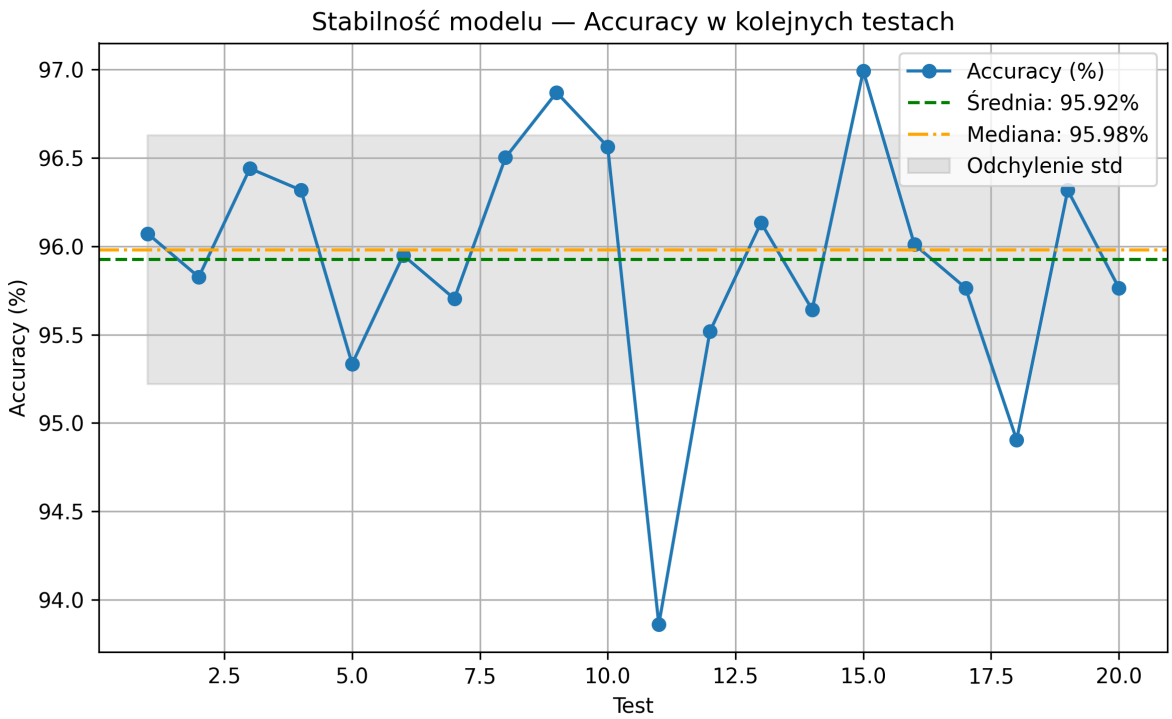


Rysunek 8: Stabilność uczenia dla batch size = 4096 – zmniejszona liczba aktualizacji wag.

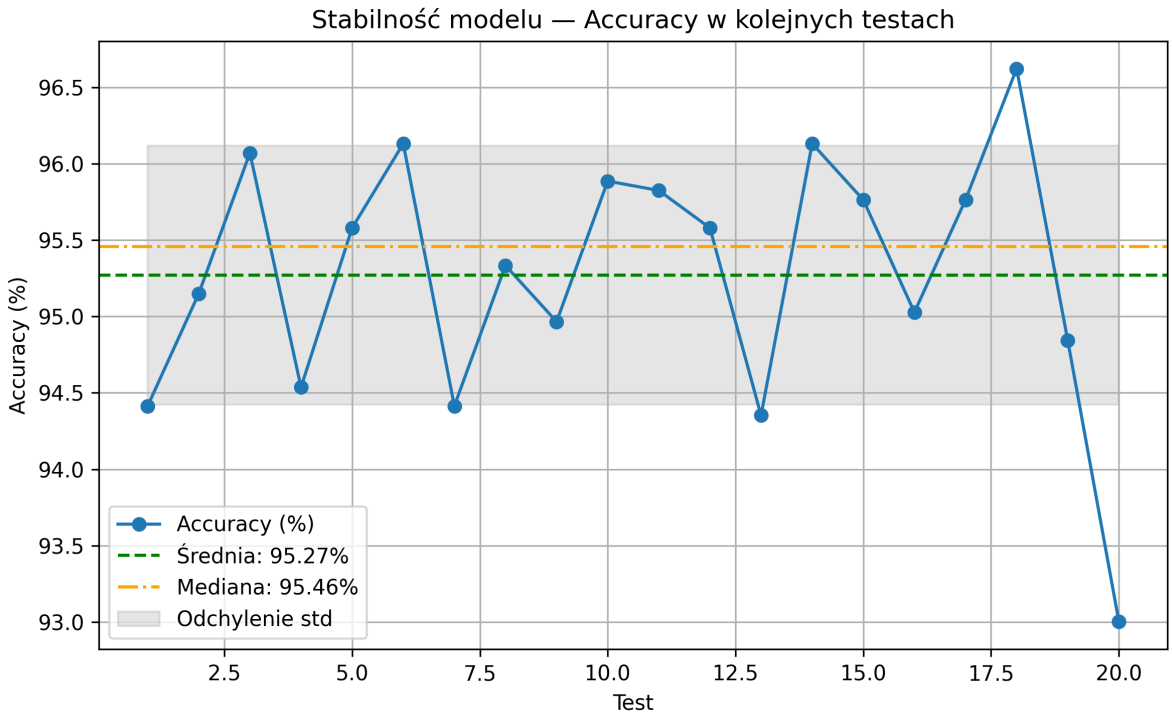
12.4 Wpływ rozdzielczości obrazu



Rysunek 9: Wpływ niskiej rozdzielczości obrazu (32x32) na jakość predykcji.

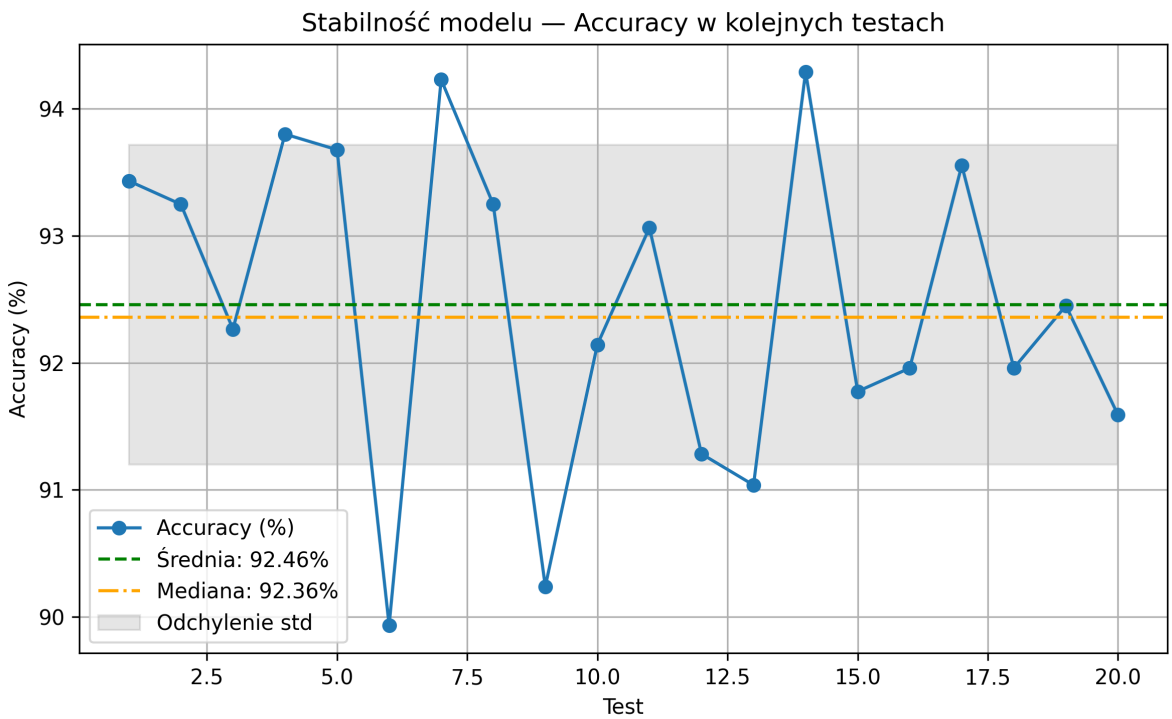


Rysunek 10: Model z rozdzielczością 64x64 – najlepszy kompromis między dokładnością a czasem.

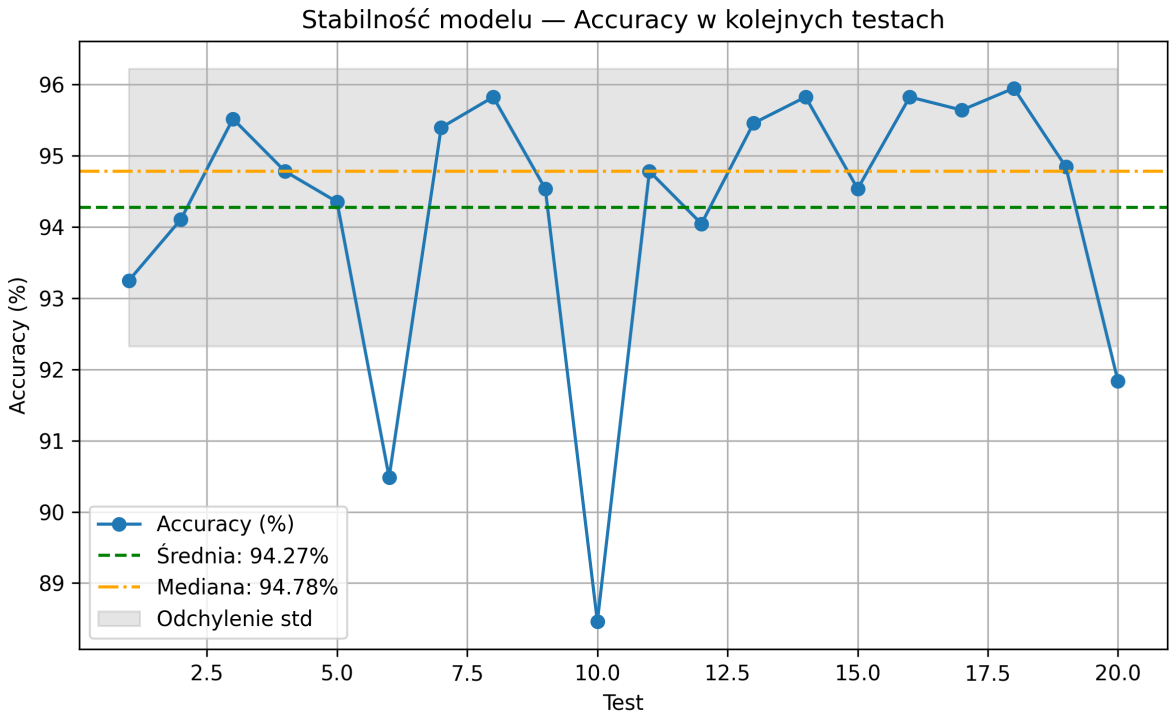


Rysunek 11: Wysoka rozdzielczość obrazu (256x256) – kosztowna czasowo, bez znacznego wzrostu dokładności.

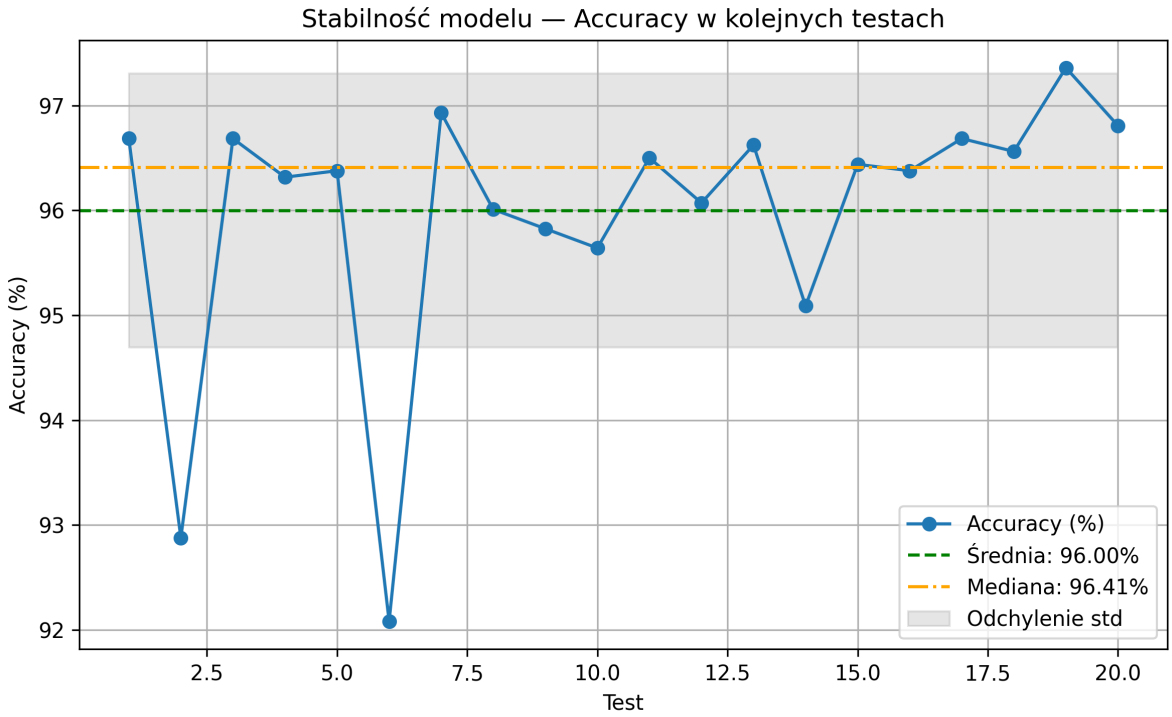
12.5 Wpływ liczby warstw konwolucyjnych



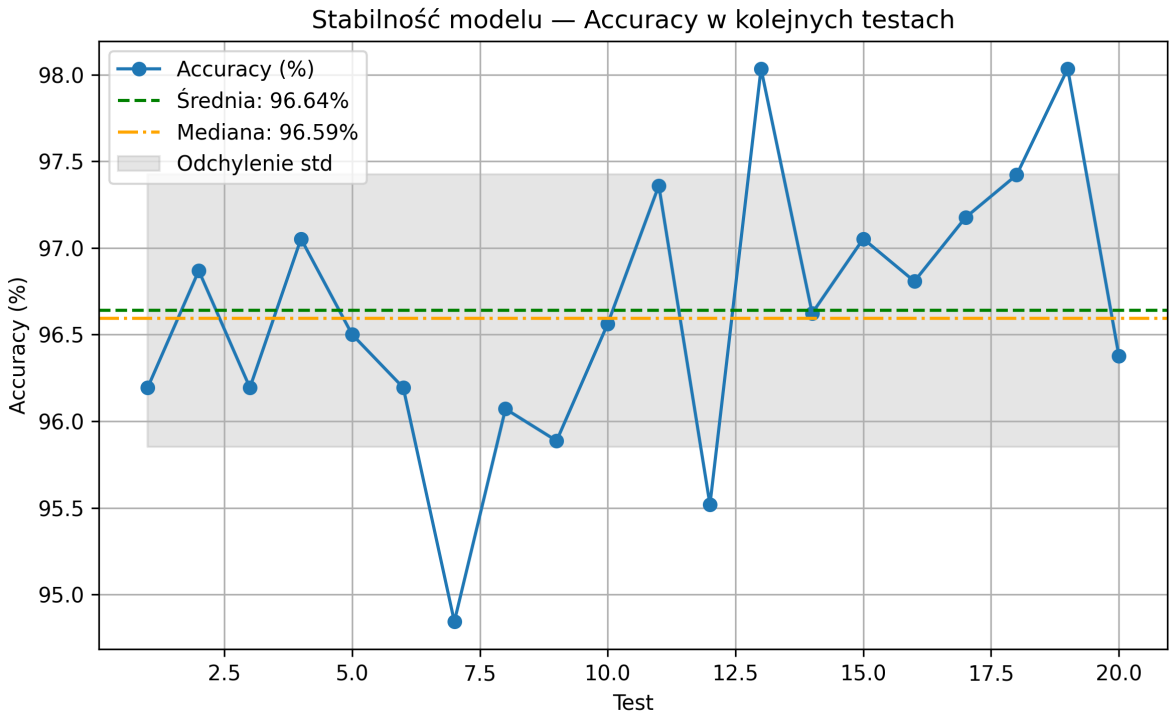
Rysunek 12: Wpływ liczby warstw – model 1-warstwowy (niska dokładność, szybki trening).



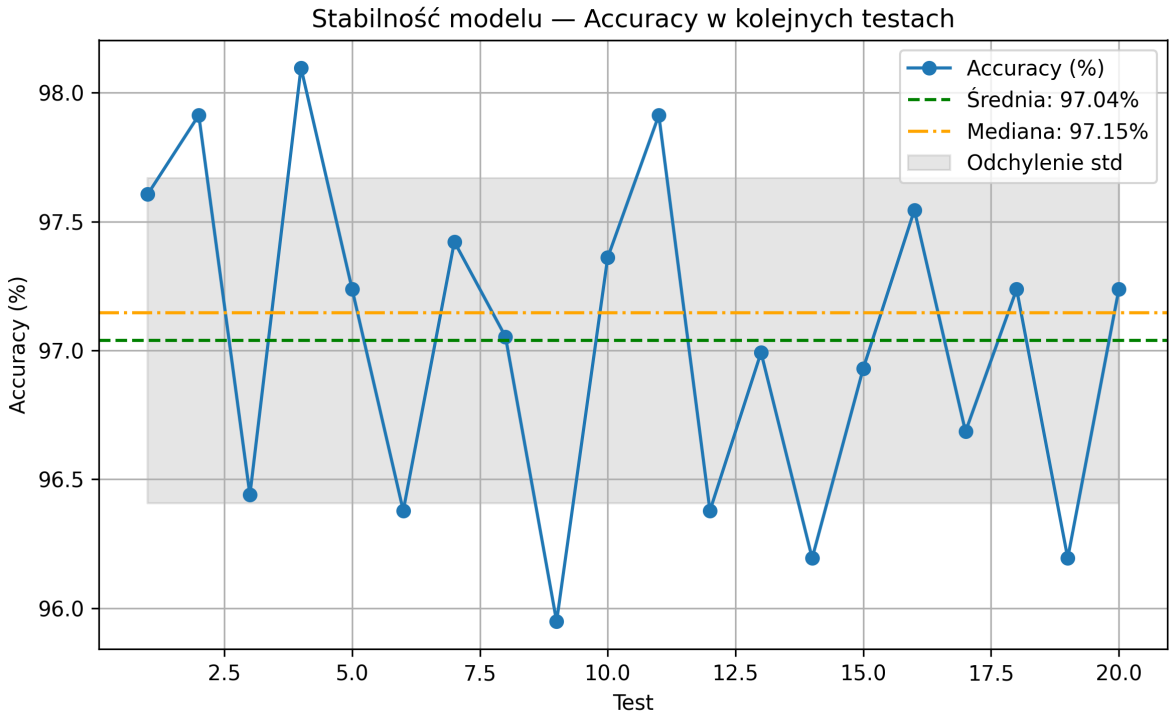
Rysunek 13: Model 2-warstwowy – wzrost dokładności w stosunku do 1 warstwy.



Rysunek 14: Model 3-warstwowy – stabilna dokładność i rozsądny czas uczenia.

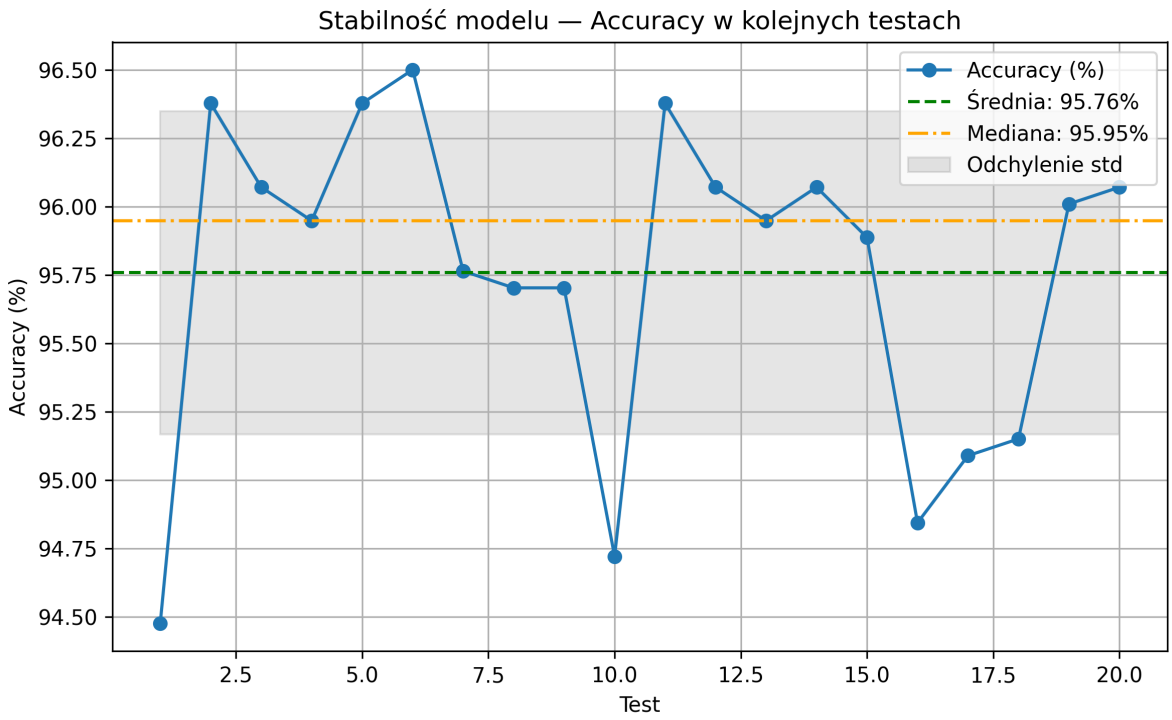


Rysunek 15: Model 4-warstwowy – dalszy wzrost dokładności kosztem czasu.



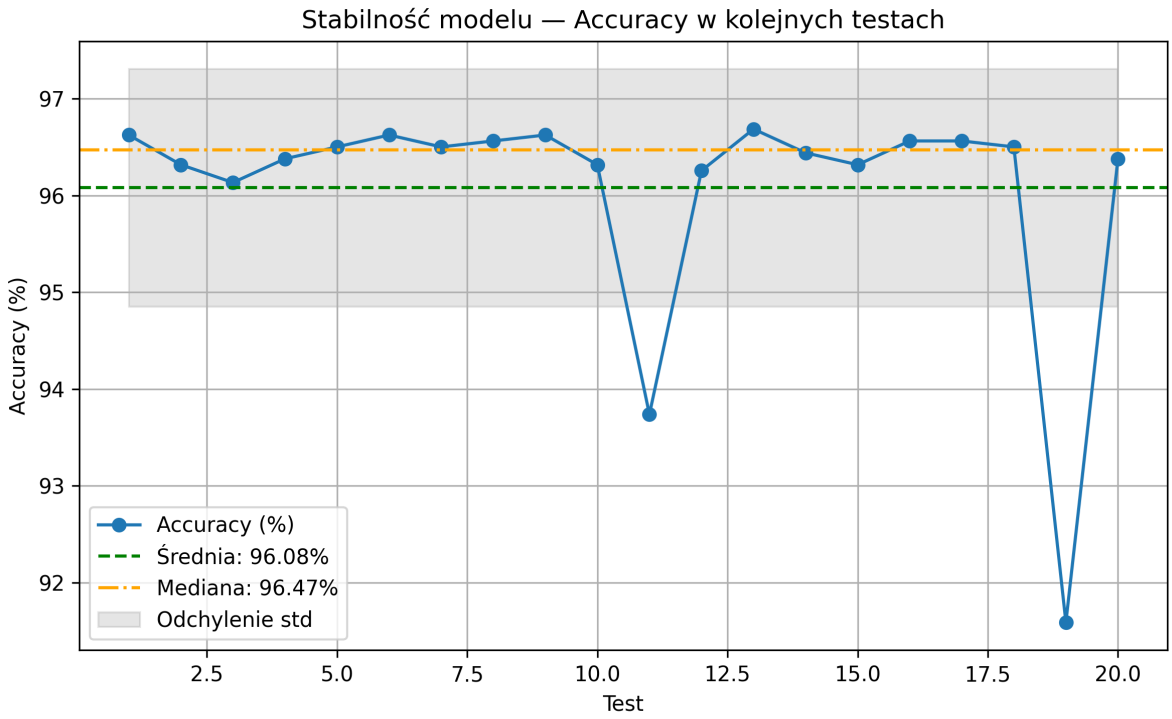
Rysunek 16: Model 5-warstwowy – najwyższa dokładność, najdłuższy czas treningu.

12.6 Wpływ szumu w danych

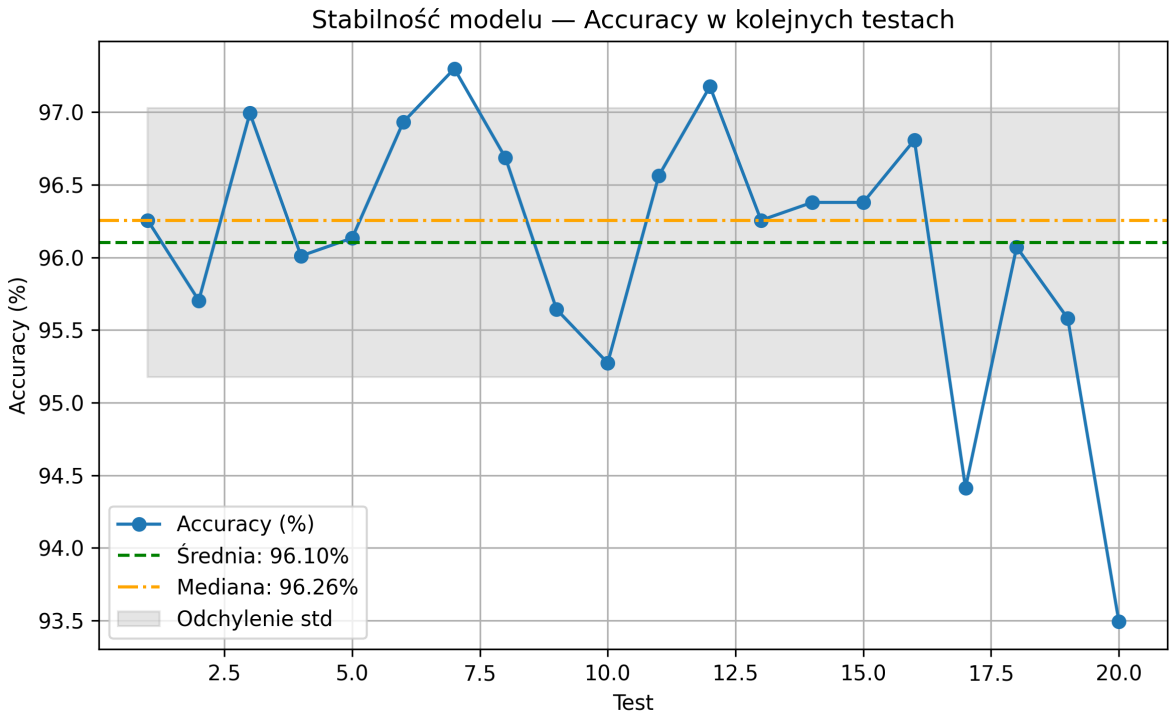


Rysunek 17: Wpływ dodania szumu do danych na dokładność predykcji.

12.7 Porównanie optymalizatorów



Rysunek 18: Użycie optymalizatora Adagrad – długi czas treningu, dobra stabilność.



Rysunek 19: Wpływ optymalizatora SGD na jakość uczenia – możliwe większe wahania dokładności.

13 Wnioski

Na podstawie przeprowadzonych badań sformułowano następujące wnioski:

- zwiększanie liczby warstw konwolucyjnych w modelu generalnie poprawia jego dokładność predykcji, jednak odbywa się to kosztem znaczącego wydłużenia czasu potrzebnego na trening,
- optymalny kompromis między osiąganą dokładnością a czasem treningu oferuje model składający się z 3 warstw konwolucyjnych, przetwarzający obrazy o rozdzielczości 64x64 pikseli, przy rozmiarze partii (batch size) wynoszącym 16,
- użycie większego rozmiaru partii (batch size) generalnie poprawia stabilność uczenia, co manifestuje się mniejszymi wahaniami dokładności między kolejnymi przebiegami. Jednakże, w kontekście osiągania zbieżności lub aktywacji mechanizmu wczesnego zatrzymywania, większe rozmiary partii mogą prowadzić do dłuższego całkowitego czasu treningu, mimo potencjalnie krótszego czasu przetwarzania pojedynczej epoki. Jest to związane z mniejszą liczbą aktualizacji wag modelu w ciągu jednej epoki,
- zastosowanie szumu gaussowskiego do danych treningowych i walidacyjnych nieznacznie obniżyło średnią dokładność modelu, ale jednocześnie zmniejszyło odchylenie standardowe, co może sugerować pewną poprawę generalizacji w trudniejszych warunkach,
- interfejs graficzny użytkownika (GUI) zaimplementowany w bibliotece PyQt6 znacząco zwiększa komfort pracy oraz ułatwia zarządzanie procesem trenowania modeli i przeprowadzanie eksperymentów, czyniąc system bardziej dostępnym dla użytkownika.

14 Załączniki

Do raportu dołączono:

- kompletny kod źródłowy aplikacji,
- pliki wynikowe eksperymentów (.csv),

- 19 szczegółowych wykresów stabilności modeli,
- pliki historii treningowej .history.json.